Guía de Octave — Sistemas de Control I (Parcial 1)

23 de septiembre de 2026

Contenido

Setup y cómo usar esta guía . . . 1
Polinomios, Laplace y fracciones parciales . . . 2
Laplace con symbolic . . . 2
Teorema del valor final . . . 2
Funciones de transferencia . . . 2
Pasar de simbólico a numérico . . . 3
Álgebra de bloques . . . 3
Ejemplo: realimentación tacométrica (guía, U2 ej. 2) . . . 3
Con parámetros literales (parcial 2018, ej. 2) . . . 4
Varias entradas: referencia y perturbación . . . 4
Equivalencia de diagramas (recuperatorio 2018, ej. 2) . . . 4
Flujo de señal y regla de Mason . . . 4
Flujo de señal a partir de una FdT . . . 4
Respuesta temporal e identificación de 1º orden . . . 5
Comandos de simulación . . . 5
Primer orden . . . 5
Segundo orden (análisis y esbozo) . . . 5
Identificación de primer orden (procedimiento de la guía, U4 ej. 8) . . . 6
De ecuaciones diferenciales a FdT (Unidad 3) . . . 6
Leyes de los elementos en Laplace . . . 6
Dos masas, varias ecuaciones (estilo guía, U3 ej. 4 y 6) . . . 7
Receta con `solve` (sistema de ecuaciones) . . . 7
Receta por sustitución (estilo cátedra, recuperatorio 2018 ej. 3) . . . 7
Verificación con valores numéricos . . . 7
Error en estado estable y estabilidad . . . 7
Numérico . . . 8
Simbólico (cuando hay parámetros) . . . 8
Con el teorema del valor final (entradas combinadas) . . . 8
Estabilidad . . . 8
Recetas por tema . . . 8
Errores comunes . . . 9

Setup y cómo usar esta guía

Alcance del parcial: Unidades 1, 2 y 3 completas, y de la Unidad 4 desde error en estado estable hasta identificación de sistemas de primer orden (incluye respuesta temporal de 1º y 2º orden). Quedan afuera identificación de 2º orden, estabilidad por Routh, lugar de raíces y espacio de estados. Los comandos y ejemplos siguen el estilo de las soluciones de la guía de la cátedra.

Hay dos “mundos” en Octave y conviene saber cuándo usar cada uno:

Paquete	Para qué	Objeto típico
`control`	Simular, graficar, calcular numéricamente (step, feedback, polos)	`G = tf(...)`

Paquete	Para qué	Objeto típico
`symbolic`	Despejar con parámetros literales (K, Kp, R, L…) y obtener expresiones	`syms s K real`

Encabezado recomendado para cada ejercicio:

```
%% Ejercicio N
close all; clear all; clc
pkg load control
pkg load symbolic
```

Si `pkg load` falla, el paquete no está instalado: `pkg install -forge control` y `pkg install -forge symbolic` (symbolic además necesita Python con SymPy). Usá `%%` para separar ejercicios en celdas y `;` al final de una línea para no imprimir el resultado.

Polinomios, Laplace y fracciones parciales

En Octave un polinomio es un vector de coeficientes en potencias decrecientes de s: $s^3 + 5s^2 + 4s + 2$ es `[1 5 4 2]`. Si falta una potencia, va un 0: $s^2 + 10s$ es `[1 10 0]`.

Comando	Qué hace	Ejemplo
`roots(p)`	Raíces del polinomio (polos si p es el denominador)	`roots([1 5 4 2])`
`poly(r)`	Polinomio a partir de sus raíces	`poly([-2 -3])` → `[1 5 6]`
`conv(a,b)`	Multiplica polinomios	`conv([1 2],[1 3])` → `[1 5 6]`
`deconv(a,b)`	Divide polinomios	`[q,r]=deconv(a,b)`
`polyval(p,x)`	Evalúa el polinomio en x	`polyval([1 5 6],0)`
`[r,p,k]=residue(num,den)`	Fracciones parciales: residuos r, polos p, término directo k	`residue(10,[1 3 2])`

Laplace con symbolic

```
pkg load symbolic
syms s t a w real
F  = laplace(exp(-2*t))          % 1/(s+2)
f  = ilaplace(10/(s*(s+2)))      % antitransformada
F7 = laplace(heaviside(t-2)*exp(-5*(t-2)))  % con retardo
```

Funciones útiles: `dirac(t)` (impulso), `heaviside(t)` (escalón), `partfrac(F, s)` (fracciones parciales simbólicas).

Teorema del valor final

$$\lim_{t\to\infty} y(t) = \lim_{s\to 0} s\,Y(s)$$

En Octave: `limit(s*Y, s, 0)` (simbólico) o `dcgain(G)` (numérico, valor final ante escalón unitario). Solo vale si el sistema es estable: todos los polos de sY(s) con parte real negativa.

Funciones de transferencia

Hay tres formas equivalentes de crear una FdT; la más cómoda para el parcial es `s = tf('s')`, porque permite escribir la FdT tal como está en el enunciado.

```
s  = tf('s');
G1 = 50/((1+0.5*s)*(1+2*s));          % forma "natural"
```

```
G2 = tf([10 10],[1 5 6]);            % num y den como vectores
G3 = zpk([-1],[-2 -3],10);           % ceros, polos, ganancia
G4 = 15/(5*s+1); G4.inputdelay = 2;   % con retardo e^(-2s)
```

Comando	Qué devuelve
`pole(G)` / `zero(G)`	Polos y ceros
`pzmap(G)`	Mapa de polos y ceros
`dcgain(G)`	Ganancia estática G(0)
`evalfr(G, x)`	G evaluada en s = x
`[num,den] = tfdata(G,'v')`	Coeficientes como vectores
`minreal(G)`	Cancela polos y ceros comunes (usalo siempre tras operar)
`damp(G)`	ωn y ζ de cada polo
`zpk(G)`	Muestra G factorizada

Pasar de simbólico a numérico

Cuando resolviste con `syms` y querés simular, reemplazá los parámetros y convertí:

```
syms s K real
Gs = K/(s*(s+2));
Gn = subs(Gs, K, 10);                % reemplaza K
[n,d] = numden(Gn);                  % numerador y denominador
G = tf(double(sym2poly(n)), double(sym2poly(d)));
```

Funciones simbólicas que usa la cátedra: `simplify`, `factor(F,'s')`, `collect(F,'s')` (agrupa por potencias de s, ideal para leer coeficientes), `pretty(F)` (imprime prolijo), `solve(eq, var)`.

Álgebra de bloques

La regla central es la realimentación negativa: la FdT de lazo cerrado es G/(1+GH). Todo diagrama se reduce combinando serie, paralelo y lazos, desde el lazo más interno hacia afuera.

$$T(s) = \frac{G(s)}{1 + G(s)H(s)}$$

Operación	Numérico (`control`)	Simbólico (`symbolic`)
Serie	`G1*G2` o `series(G1,G2)`	`G1*G2`
Paralelo	`G1+G2` o `parallel(G1,G2)`	`G1+G2`
Realim. negativa	`feedback(G,H)`	`G/(1+G*H)`
Realim. unitaria	`feedback(G,1)`	`G/(1+G)`
Realim. positiva	`feedback(G,H,+1)`	`G/(1-G*H)`

Ejemplo: realimentación tacométrica (guía, U2 ej. 2)

```
s = tf('s');
Kp = 31; Kd = 16;
G1 = 53*(s+1)/((s+10)*(s+100));
G2 = 1/s;
Faux  = feedback(G1, Kd);       % lazo interno (tacómetro)
FdTLA = Kp*Faux*G2;             % lazo abierto
FdTLC = feedback(FdTLA, 1);     % lazo cerrado
step(FdTLC); grid
```

Con parámetros literales (parcial 2018, ej. 2)

```
syms s Kd Kp real
G   = 20/(s^2+10*s);
Gd  = Kd*s;
GLC = collect(G/(1+G*Gd), 's');  % lazo interno
GLA = Kp*GLC                      % lazo abierto del externo
```

Varias entradas: referencia y perturbación

Con R(s) y N(s), aplicá superposición: una FdT por entrada, anulando la otra. Si N entra sumando después de G2 en la cadena directa G1·G2 con realimentación H:

```
syms s G1 G2 H real
T_R = G1*G2/(1+G1*G2*H);   % Y/R con N = 0
T_N = 1/(1+G1*G2*H);       % Y/N con R = 0 (depende de dónde entra N)
```

El numerador es siempre la ganancia del camino directo desde esa entrada hasta Y; el denominador (1 + ganancia de lazo) es el mismo para todas. Si te piden “influencia de una ganancia” (p. ej. KD), mirá cómo aparece en T_R y T_N y usá `limit(..., KD, inf)` para ver qué pasa cuando crece.

Equivalencia de diagramas (recuperatorio 2018, ej. 2)

Para hallar un bloque G* que haga equivalentes dos diagramas, escribí la FdT de cada uno en función de G*, igualalas y despejá:

```
syms s Gx G H real
T1 = G/(1+G*H);            % diagrama original (ejemplo)
T2 = Gx*G/(1+G);           % diagrama con G* (ejemplo)
Gx = simplify(solve(T1 == T2, Gx))
```

Las FdT T1 y T2 del bloque anterior son ilustrativas: armalas según el diagrama del enunciado.

Flujo de señal y regla de Mason

Mason da la FdT de un diagrama de flujo de señal sin reducirlo paso a paso: sumás las ganancias de los caminos directos, cada una multiplicada por su cofactor, y dividís por el determinante del gráfico.

$$T = \frac{\sum_k M_k \, \Delta_k}{\Delta}, \qquad \Delta = 1 - \sum L_i + \sum L_i L_j - \sum L_i L_j L_k + \dots$$

- Mk: ganancia del camino directo k (producto de las ramas de entrada a salida, sin pasar dos veces por un nodo).
- Li: ganancia de cada lazo; LiLj: productos de pares de lazos que no se tocan (sin nodos en común); y así con ternas.
- Δk: el Δ calculado solo con los lazos que no tocan al camino k (si todos lo tocan, Δk = 1).

En Octave se cargan a mano los caminos y lazos, y Octave hace el álgebra (guía, U2 ej. 10):

```
s = tf('s');
l1 = -1/s;  l2 = -1/s;                % lazos
l3 = -5*s*30*1/s*1/s;  l4 = -5*s*30*s;  l5 = 5;
Det = 1 - (l1 + l2 + l3 + l4 + l5);   % sumar pares no tocantes si los hay
M1 = 1/s*1/s*30;  M2 = 5*30;          % caminos directos
D1 = 1;  D2 = 1;                      % cofactores
T = minreal((M1*D1 + M2*D2)/Det)
```

La versión simbólica es igual, con `syms G1 G2 H1 real` y `simplify` al final. Para demostrar que dos diagramas son (o no) equivalentes, calculá ambas FdT y verificá `simplify(T1 - T2) == 0`.

Flujo de señal a partir de una FdT

Para dibujar el diagrama de una FdT (guía, U2 ej. 12): dividí numerador y denominador por la mayor potencia de s, introducí una variable auxiliar Q(s), igualá numeradores y denominadores,

despejá Q(s) y dibujá la cadena de integradores 1/s con los lazos de realimentación que dan los coeficientes del denominador.

Respuesta temporal e identificación de 1º orden

De la Unidad 4 entra la respuesta temporal de 1º y 2º orden (esbozar la respuesta, calcular K, τ, ζ, ωn, ts) y la identificación de sistemas de primer orden a partir de su respuesta al escalón. La identificación de 2º orden no entra.

Comandos de simulación

Comando	Uso
`step(G)` / `step(G, Tfinal)`	Respuesta al escalón unitario; Tfinal fija el eje de tiempo
`step(A*G, Tfinal)`	Escalón de amplitud A
`step(G1, G2)`	Dos respuestas superpuestas
`[y,t] = step(G)`	Devuelve los datos sin graficar
`impulse(G)`	Respuesta al impulso
`lsim(G, u, t)`	Entrada arbitraria; rampa: `t=0:0.01:10; lsim(G,t,t)`
`pzmap(G)`, `damp(G)`	Mapa de polos y ceros; ωn y ζ de cada polo
`grid on`, `hold on`, `figure`	Formato del gráfico

Primer orden

$$G(s) = \frac{K}{\tau s + 1} e^{-t_d s}$$

Dato	Cómo se lee o calcula
K	valor final / amplitud del escalón
τ	tiempo en que la salida llega al 63.2 % del valor final (desde el fin del retardo)
ts (2 %)	4τ; la cátedra también usa ts ≈ 5τ como “establecimiento al 100 %”
td	tiempo muerto antes de que la salida empiece a moverse; no se cuenta en ts
Polo	$s = -1/\tau$

Segundo orden (análisis y esbozo)

$$G(s) = \frac{K\,\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

De la FdT: ωn = √(término independiente), ζ = coeficiente de s / (2ωn), K = G(0). El tipo de respuesta depende de ζ: ζ = 0 oscilatoria sin amortiguar, 0 < ζ < 1 subamortiguada (con sobrepaso), ζ = 1 críticamente amortiguada, ζ > 1 sobreamortiguada (dos polos reales, como dos 1º orden en serie).

$$M_p = e^{-\zeta\pi/\sqrt{1-\zeta^2}}, \qquad t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}}, \qquad t_s(2\%) \approx \frac{4}{\zeta\omega_n}$$

```
%% Motor CC (guía U3 ej. 2 / U4 ej. 5)
s = tf('s');
W_V = 3000/(s^2 + 156.25*s + 1837.5);
K    = dcgain(W_V)                % ganancia estática
wn   = sqrt(1837.5)
zeta = 156.25/(2*wn)              % > 1: sobreamortiguado
roots([1 156.25 1837.5])          % polos
step(10*W_V); grid                % escalón de 10 V
```

Identificación de primer orden (procedimiento de la guía, U4 ej. 8)

1. Leer el valor de régimen yss en la gráfica.
2. Calcular 0.98·yss (o 0.632·yss para τ directo) y ubicar en la gráfica el tiempo en que la salida lo alcanza.
3. τ = ts/4 (o el tiempo al 63.2 %). Si hay retardo, restarlo antes.
4. K = yss / amplitud del escalón.
5. Armar la FdT, simular con el mismo Tfinal de la gráfica y comparar puntos.

```
%% Identificación de 1º orden
close all; clear all; clc
pkg load control
A   = 1;        % amplitud del escalón
yss = 8500;     % valor de régimen leído
ts  = 40;       % tiempo al 98 % (sin contar el retardo)
td  = 0;        % retardo
Tf  = 90;       % tiempo final de la gráfica
K   = yss/A;
tau = ts/4;
G = tf(K, [tau 1]); G.inputdelay = td;
step(A*G, Tf); grid on
fprintf('0.98*yss = %.2f\n', 0.98*yss)
```

Ojo: en Octave `log` es logaritmo natural (ln); `log10` es el decimal.

De ecuaciones diferenciales a FdT (Unidad 3)

El método es siempre el mismo: transformar cada ecuación con condiciones iniciales nulas (d/dt → s), escribir las variables como símbolos y eliminar las intermedias con `solve` hasta quedarte con salida/entrada.

Leyes de los elementos en Laplace

Sistema	Elemento	Ecuación en s	Impedancia / término
Eléctrico	Resistencia R	V = R·I	R
Eléctrico	Inductor L	V = L·s·I	sL
Eléctrico	Capacitor C	V = I/(s·C)	1/(sC)
Mecánico traslación	Masa M	F = M·s²·X	Ms²
Mecánico traslación	Amortiguador B	F = B·s·X	Bs
Mecánico traslación	Resorte K	F = K·X	K
Mecánico rotación	Inercia J	T = J·s·Ω	Js (con Ω = sθ)
Mecánico rotación	Fricción viscosa B	T = B·Ω	B
Motor CC	Acople electromecánico	T = Ka·I; fcem = Kb·Ω	Ka, Kb

Circuitos: con impedancias, un divisor RC da Vo/Vi = 1/(RCs+1); en redes de varias mallas planteá las ecuaciones de malla o nodo y despejá con `solve` (guía, U2 ej. 1). Sistemas mecánicos: una ecuación de Newton por masa (diagrama de cuerpo libre), con los resortes y amortiguadores entre masas actuando sobre la diferencia de posiciones (X1 − X2).

Dos masas, varias ecuaciones (estilo guía, U3 ej. 4 y 6)

```
pkg load symbolic
syms s M1 M2 B1 K1 K2 real
syms X1 X2 U
eq1 = M1*s^2*X1 + B1*s*(X1-X2) + K1*(X1-X2) == U;
eq2 = M2*s^2*X2 + B1*s*(X2-X1) + K1*(X2-X1) + K2*X2 == 0;
S = solve(eq1, eq2, X1, X2);        % S.X1, S.X2
G1 = simplify(S.X1/U)               % X1(s)/U(s)
G2 = simplify(S.X2/U)               % X2(s)/U(s)
G1n = subs(G1, {M1 M2 B1 K1 K2}, {1 2 0.5 10 5})  % valores numéricos
```

Las ecuaciones y valores del bloque anterior son ilustrativos: armalos a partir del diagrama de cuerpo libre del enunciado. Si hay un sistema no lineal (p. ej. péndulo invertido), primero se linealiza alrededor del punto de operación (sen θ ≈ θ, cos θ ≈ 1) y después se transforma.

Receta con `solve` (sistema de ecuaciones)

Ejemplo con un motor de CC genérico: E = R·I + L·s·I + Kb·W y J·s·W = Ka·I − B·W (con TL = 0).

```
pkg load symbolic
syms s R L Kb Ka J B real
syms E I W
eq1 = E == R*I + L*s*I + Kb*W;
eq2 = J*s*W == Ka*I - B*W;
sol = solve([eq1, eq2], [I, W]);   % despeja las incógnitas
G_WE = simplify(sol.W / E);         % W(s)/E(s)
pretty(collect(G_WE, 's'))
```

Regla: en `solve` ponés como incógnitas todas las variables menos la entrada. La salida queda en función de la entrada y dividís.

Receta por sustitución (estilo cátedra, recuperatorio 2018 ej. 3)

```
syms R L Kb J Ka B Kce N Is s real
Qs = Ka/(J*s^2 + B*s + Kce/N)*Is;   % de la ec. mecánica
Es = R*Is + L*s*Is + Kb*s*Qs;       % de la ec. eléctrica
Xs  = simplify(Es/Is);              % E/I
Mot = Ka*collect(1/Xs, 's');        % Tm/E = Ka*I/E
pretty(Mot)
```

Verificación con valores numéricos

Una vez obtenida la FdT simbólica, reemplazá valores con `subs` y simulá (ver “Pasar de simbólico a numérico”). Si el ejercicio pide diagrama de bloques, cada ecuación transformada es un bloque: la ecuación eléctrica da 1/(Ls+R), la mecánica 1/(Js+B), y los términos Kb y Ka son ganancias en el lazo.

Error en estado estable y estabilidad

Para realimentación unitaria, el error depende del tipo del sistema (cantidad de polos en el origen de la FdT de lazo abierto G) y de las constantes de error. Antes de calcular, verificá que el lazo cerrado sea estable; si no, el ess no existe.

Entrada	Constante	ess
Escalón A/s	Kp = lim G(s), s→0	A/(1+Kp)
Rampa A/s²	Kv = lim s·G(s), s→0	A/Kv
Parábola A/s³	Ka = lim s²·G(s), s→0	A/Ka

Tipo 0: error finito al escalón. Tipo 1: cero al escalón, finito a la rampa. Tipo 2: cero al escalón y a la rampa, finito a la parábola.

Numérico

```
s = tf('s');
G  = 50/((1+0.5*s)*(1+2*s));
Kp = dcgain(G)
Kv = dcgain(minreal(s*G))
Ka = dcgain(minreal(s^2*G))
ess_escalon = 1/(1+Kp)
ess_rampa   = 1/Kv
step(feedback(G,1)); grid     % verificación visual
```

Simbólico (cuando hay parámetros)

Parcial 2018, ej. 2: lazo interno con Kd·s y ganancia Kp; con Kd = 10 se busca ess ≤ 0.1 a la rampa.

```
syms s Kd Kp real
G   = 20/(s^2+10*s);
GLA = simplify(Kp*G/(1+G*Kd*s));   % = 20Kp/(s(s+10+20Kd))
Kv  = limit(s*GLA, s, 0)           % 20Kp/(10+20Kd)
ess = subs(1/Kv, Kd, 10)           % 21/(2Kp)
solve(ess == 0.1, Kp)              % Kp = 105 → ess ≤ 0.1 si Kp ≥ 105
```

Con el teorema del valor final (entradas combinadas)

Para r(t) = 1 + 5t, o cuando la realimentación no es unitaria, calculá E(s) directamente:

```
syms s K real
R = 1/s + 5/s^2;
G = K*(2*s+1)/(s*(4*s+1)*(s+1)^2);
E = R/(1+G);                % realim. unitaria
ess = limit(s*E, s, 0)      % 5/K
```

Si la realimentación es H(s) ≠ 1, el error es E = R − Y = R·(1 − T), con T la FdT de lazo cerrado.

Estabilidad

Un sistema es estable si todos los polos de lazo cerrado tienen parte real negativa. Comprobación rápida:

```
T = feedback(G, 1);
pole(T)                  % todos con real < 0
all(real(pole(T)) < 0)   % 1 = estable
```

Para este parcial alcanza con justificar la estabilidad por la ubicación de los polos (lo que piden, por ejemplo, U3 ej. 1 y 8). Routh y estabilidad relativa no entran.

Recetas por tema

Cada tema del parcial tiene una herramienta principal en Octave; la tabla sirve para decidir por dónde arrancar un ejercicio.

Unidad	Tema	Herramienta principal
U1	Transformada de Laplace, TVF	`laplace`, `ilaplace`, `limit(s*F,s,0)`
U2	FdT de circuitos	impedancias + `solve`
U2	Álgebra de bloques	`feedback` o G/(1+GH) simbólico
U2	Flujo de señal y Mason	caminos, lazos y Δ cargados a mano
U3	Modelado de sistemas físicos	`syms` + `solve` + `subs`
U4	Error en estado estable	`dcgain`, `limit`, `solve` para la ganancia

Unidad	Tema	Herramienta principal
U4	Respuesta temporal 1º y 2º orden	`step`, `damp`, fórmulas de ζ, ωn
U4	Identificación de 1º orden	K, τ desde la gráfica + `step`

1. **Laplace:** definí `syms s t real`, transformá con `laplace`, volvé con `ilaplace`; con fracciones parciales usá `residue` (numérico) o `partfrac` (simbólico). Valor final con TVF solo si es estable.
2. **Circuitos:** reemplazá cada elemento por su impedancia; planteá mallas o nodos; `solve` para la tensión de salida; dividí por la entrada.
3. **Bloques:** del lazo más interno al externo con G/(1+GH); serie multiplica, paralelo suma; con perturbación, una FdT por entrada.
4. **Mason:** listá caminos directos, lazos y pares no tocantes; armá Δ y los Δk; verificá contra la reducción por bloques si el tiempo alcanza.
5. **Modelado:** diagrama de cuerpo libre o circuito; ecuaciones en s con CI nulas; `solve` sobre todas las variables salvo la entrada; `simplify` y `collect(...,'s')`; `subs` con valores y simular.
6. **Error en estado estable:** FdT de lazo abierto, tipo del sistema, constante según la entrada; si piden un valor de ganancia, desigualdad y `solve`. Verificá con `step` o `lsim`.
7. **Respuesta temporal:** de la FdT sacá K, τ o ωn y ζ; clasificá la respuesta por ζ; calculá ts (y Mp, tp si es subamortiguado); verificá con `step`.
8. **Identificación de 1º orden:** yss, ts al 98 % (o τ al 63.2 %), retardo, K; FdT y simulación con el mismo Tfinal que la gráfica.

Errores comunes

La mayoría de los errores en Octave con este tema vienen de mezclar los mundos simbólico y numérico, o de sintaxis de polinomios.

Síntoma	Causa	Solución
`'tf' undefined`	Falta cargar el paquete	`pkg load control`
`'syms' undefined`	Falta symbolic	`pkg load symbolic`
`step` no acepta la FdT	Es simbólica (`sym`), no `tf`	Convertir con `numden` + `sym2poly`
`0.6s` da error	Falta el `*`	Escribir `0.6*s`
FdT con orden de más	Polos y ceros sin cancelar	`minreal(G)`
`dcgain` da Inf	Hay polo en el origen (integrador)	Es esperable: usá `s*G` para Kv
Mp o ζ imaginario	Mp leído en % (25) en vez de fracción (0.25)	Mp = (ymax−yss)/yss
Curva simulada no coincide en escala	Tfinal distinto al de la gráfica	`step(G, Tfinal)`
Signo mal en el lazo	`feedback` asume negativa	`feedback(G,H,+1)` para positiva
`ans` con fracciones enormes	Resultado simbólico sin simplificar	`simplify`, `factor`, o `double(...)`

Chequeos finales antes de escribir en tinta: la FdT tiene el mismo orden que el número de elementos que almacenan energía; el valor final simulado coincide con `dcgain`; el ess calculado coincide con la diferencia entre entrada y salida en la simulación (`lsim` para rampa).